

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №5
Реализация и описание паттернов проектирования
тема

Преподаватель

подпись, дата

Д. В. Грузенкин
инициалы, фамилия

Студент КИ23-17/1Б, 032320072
номер группы, зачетной книжки

подпись, дата

М. А. Мальцев
инициалы, фамилия

Красноярск 2026

1 Цель

Выбрать тему для системы, архитектура которой будет разрабатываться в течение семестра, дать ей название и описать назначение.

Провести сбор и анализ требований к системе, архитектура которой будет разрабатываться в этом семестре.

Выбрать архитектурный паттерн для проектируемой системы и обосновать выбор.

Описать систему с помощью методологии DDD и C4.

Реализовать паттерны проектирования данной системы.

2 Задачи

Для выполнения практической работы необходимо выполнить следующие задачи:

- придумать систему, ее название и назначение;
- построить UML-диаграмму вариантов использования;
- предоставить отчет с выполненными заданиями, ответить на вопросы и выполнить дополнительные задания по усмотрению преподавателя.

Составить список требований для проектируемой системы на следующих уровнях:

- бизнес-требования (для их составления вам понадобится пусть и вымышленный, но конкретный заказчик);
- требования пользователей;
- функциональные требования;
- нефункциональные требования.

Описать источники и методы получения требований, которые бы вы использовали в реальной работе с заказчиком по проектируемой системе.

Выбрать один из приведённых в лекции архитектурных паттернов (или любой другой на своё усмотрение) и реализовать в коде.

Бизнес-логику приложения писать не нужно. Реализуем только связи между классами и модулями, для сервисной и микросервисной архитектуры

будет достаточно только структуры проектов (приложений/сервисов) и связей между ними.

В отчёте необходимо обосновать выбор реализованного шаблона в виде ADR (Architecture Decision Record).

Необходимо реализовать карту контекстов по методологии DDD, в соответствии с выбранным вами архитектурным шаблоном. Для смыслового ядра и одной вспомогательной подобласти также необходимо привести диаграмму контекста нотации C4. Необходимо описать каждый из приведенных рисунков, а не просто их вставить.

Необходимо реализовать в коде три паттерна проектирования (один на каждый вид паттернов). Для написанного кода построить UML-диаграмму классов, а также привести обоснование выбора реализованных паттернов проектирования и для каждого вида привести пример паттерна, который было бы использовать нецелесообразно.

3 Ход выполнения

3.1 Выбор и описание системы для проектирования

В качестве системы для проектирования был выбран кейс с Хакатона, для которого нужно было разработать удобный инструмент для работников металлопрокатного предприятия, с помощью которого они смогут быстро фиксировать дефекты на листах металла.

Была придумана система MetalDefectTracker (MDT), представляющая из себя веб-ориентированную систему автоматического контроля качества поверхности листового металла на основе компьютерного зрения с ролевым доступом (оператор, технолог, администратор).

Она получает изображения листа металла с камеры, предобрабатывает их с помощью обученной модели для детекции 6 разных типов дефектов и предоставляет оператору интерфейс для верификации результатов. Все загружаемые изображения сохраняются в базе. Если дефекты обнаружены, то система помечает его статусом (pending), требующим действий от оператора.

Оператор может подтверждать или отклонять обнаруженные дефекты, а также вносить ручные исправления: выбирать, какие дефекты из найденных исключить, самостоятельно выделять прямоугольником не обнаруженные моделью дефекты и давать им одну из семи меток типа (в том числе «другое»). Все действия фиксируются, накапливается статистика по работе операторов и сайта, доступная для анализа технологам и администраторам. Оператор может просматривать историю действий с поступающими изображениями, фильтровать по статусу, по наличию дефектов или по их типам, а также смотреть свою статистику (количество обработанных изображений, количество обнаруженных дефектов и среднее время на верификацию после поступления фото). Технолог может только просматривать общую статистику всех операторов и изображений, или смотреть статистику определенного оператора. Администратор имеет права и оператора, и технолога, но также имеет админ панель, позволяющую ему управлять пользователями системы, а также настраивать камеру, с которой поступают изображения (частота снимков, разрешение), и модель детекции (группировать фотографии с дефектами, помеченными как «другое», давать новую метку и производить дообучение).

UML-диаграмма вариантов использования представлена на рисунках 1 и 2.

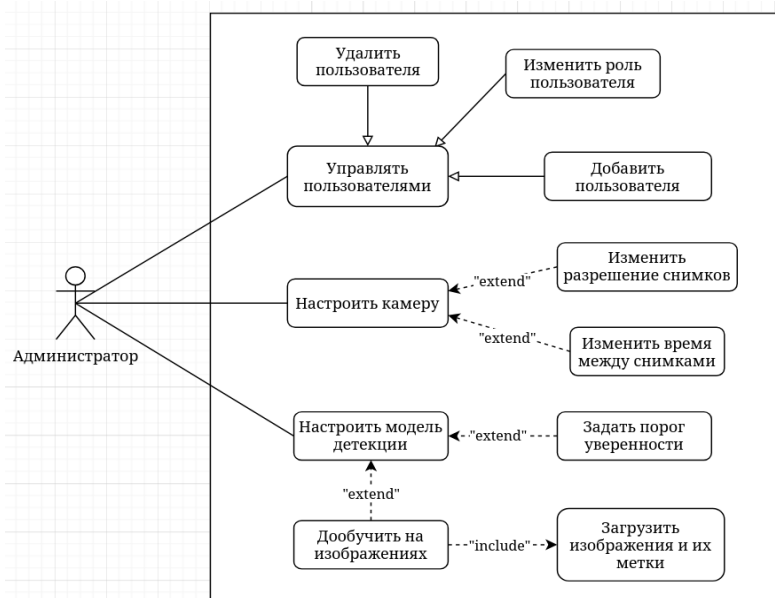


Рисунок 1 – UML-диаграмма вариантов использования системы MDT (только для роли «Администратор»)

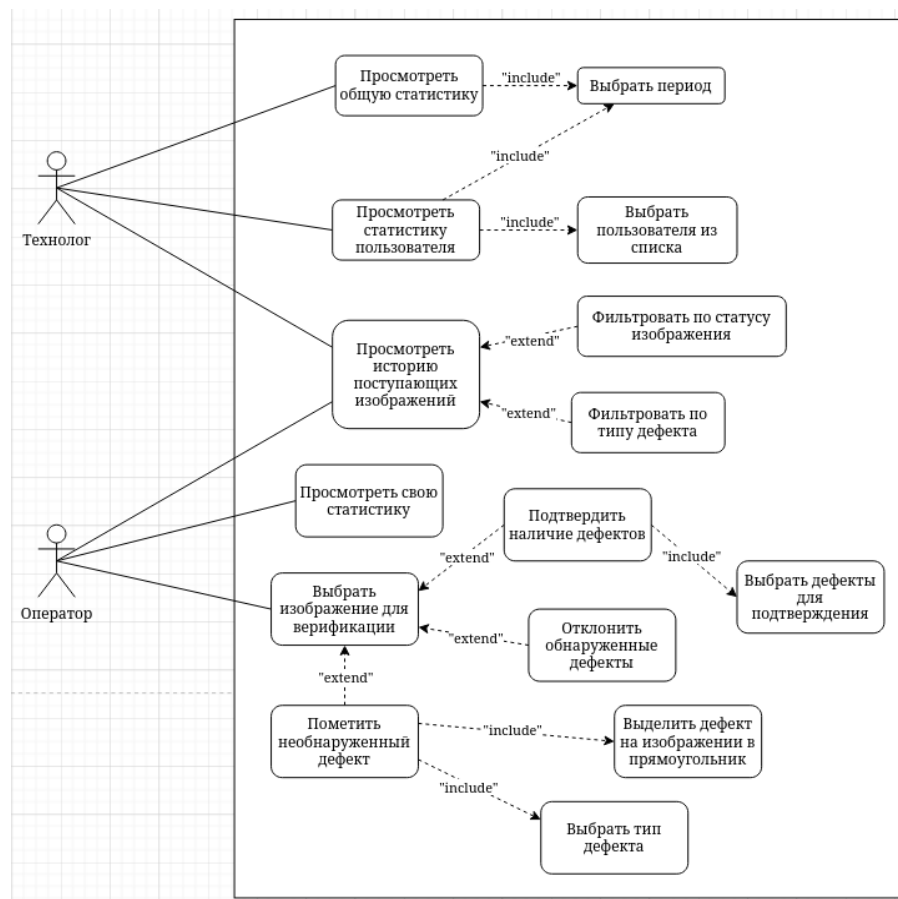


Рисунок 2 – UML-диаграмма вариантов использования системы MDT (только для ролей «Технолог» и «Оператор»)

3.2 Составление списка бизнес-требований

В качестве заказчика выступает вымышленное металлопрокатное предприятие с отделом контроля качества (ОТК) и технологической службой, расположенное в Сибири.

Для формирования бизнес-требования необходимо воспользоваться методами получения требования. Одним из таким методов является проведение интервью с заказчиком. Было проведено вымышленное интервьюирование начальника металлопрокатного предприятия с помощником в лице главного технолога, которое также было оформлено по протоколу, представленному далее.

Проект: «Система автоматизированного контроля качества поверхности листового металла»

Дата проведения: 21.02.2026

Интервьюер: Мальцев Максим Александрович

Интервьюируемые:

- Стейкхолдер 1 – Иванов Иван Иванович, начальник ОТК
- Стейкхолдер 2 – Иванова Ольга Ивановна, главный технолог

Проблематика: Автоматизация и цифровизация контроля качества поверхности листового металла на металлургическом предприятии в Сибири

Вопрос № 1:

Тайминг вопроса: 00:02-00:07

Текст вопроса: Как в настоящий момент организован контроль качества поверхности листового металла?

Таймкод: 00:03

Ответы на вопрос:

Стейкхолдер 1 – Сейчас контроль осуществляется визуально операторами ОТК на выходе с прокатной линии. Каждый лист осматривается вручную. Оператор принимает решение – годен лист или брак. При обнаружении дефектов он фиксирует их в журнале и иногда делает фотографии. Процесс трудоемкий, зависит от опыта и внимательности сотрудника.

Стейкхолдер 2 – С технологической точки зрения нас не устраивает высокая вариативность результатов между сменами. При больших объемах производства у людей устают глаза, и часть дефектов может быть пропущена.

Вопрос № 2:

Тайминг вопроса: 00:07-00:13

Текст вопроса: Какие основные проблемы вы видите в текущем процессе контроля?

Таймкод: 00:08

Ответы на вопрос:

Стейкхолдер 1 – Во-первых, это время. Проверка одного листа занимает до нескольких минут, а поток большой. Во-вторых, человеческий фактор –

усталость, невнимательность. Это приводит либо к пропуску дефектов, либо к излишнему браку.

Стейкхолдер 2 – Нам важно повысить стабильность качества. Сейчас результаты зависят от конкретного оператора и смены. Нет полной повторяемости.

Вопрос № 3

Тайминг вопроса: 00:13-00:18

Текст вопроса: Какие цели вы ставите перед системой автоматического контроля?

Таймкод: 00:14

Ответы на вопрос:

Стейкхолдер 1 – Система должна автоматически находить дефекты на поверхности листа и предлагать оператору результаты для быстрой верификации. Оператор должен подтверждать статус листа – годен или брак – быстрее, чем сейчас.

Стейкхолдер 2 – Мы хотим сократить время инспекции без потери качества, а лучше – с его повышением. Автоматическая первичная детекция должна стать фильтром, а человек – подтверждающим звеном.

Вопрос № 4

Тайминг вопроса: 00:18-00:24

Текст вопроса: Какие типы дефектов критичны для вашего производства?

Таймкод: 00:19

Ответы на вопрос:

Стейкхолдер 2 – Сейчас у нас на производстве основными типами дефектов поверхности металлов считаются: окалина (с меткой RS), пятна (Pa), трещины (Cr), шероховатая поверхность (PS), включения (In) и царапины (Sc). Нам важно, чтобы классификация была стандартизирована. Чтобы все смены пользовались единым справочником и одинаковыми критериями оценки.

Вопрос № 5

Тайминг вопроса: 00:24-00:31

Текст вопроса: Насколько для вас важна прослеживаемость и хранение истории контроля?

Таймкод: 00:25

Ответы на вопрос:

Стейкхолдер 2 – Это критично. Мы должны иметь возможность по любой партии быстро поднять: фотографии, найденные дефекты, кто и когда принял решение, были ли изменения разметки. Сейчас это разрозненные данные, а нам требуется единая доказательная база. Мы должны показать, как проводился контроль, какие дефекты были обнаружены, как принималось решение.

Вопрос № 6

Тайминг вопроса: 00:31-00:38

Текст вопроса: Как технологическая служба планирует использовать данные автоматического контроля?

Таймкод: 00:32

Ответы на вопрос:

Стейкхолдер 2 – Нам нужна статистика по типам дефектов, их частоте, распределению по сменам и участкам. Это позволит выявлять причины появления дефектов, анализировать технологические режимы и снижать дефектность.

Вопрос № 7

Тайминг вопроса: 00:38–00:45

Текст вопроса: Какой эффект вы ожидаете после внедрения системы?

Таймкод: 00:40

Ответы на вопрос:

Стейкхолдер 1 – Сокращение трудозатрат операторов, ускорение контроля, снижение процента пропущенных дефектов.

Стейкхолдер 2 – Повышение стабильности качества, управляемость процесса, аналитика причин дефектов и снижение общей дефектности продукции.

По результатам проведенного интервью был сформулирован список бизнес-требований, каждый пункт списка содержит цели предприятия и ожидаемый эффект от возможного решения.

Бизнес-требования заказчика:

1. Сократить трудозатраты и время контроля качества поверхности листового металла – система должна уменьшить время инспекции листа за счет автоматической первичной детекции дефектов и удобной верификации оператором статуса продукта (годен или брак).

2. Снизить долю пропущенных дефектов и повысить стабильность качества – система должна обеспечить полноту и повторяемость выявления дефектов, уменьшив влияние человеческого фактора (невнимательности, усталости).

3. Обеспечить прослеживаемость контроля качества и доказательную базу – система должна предоставить возможность быстро получить информацию по контролю партии: фото, найденные дефекты, решения оператора, изменения разметки, кто и когда подтвердил или исправил; чтобы быстро формировать отчетность и проводить разбор инцидентов.

4. Дать технологам инструмент для анализа причин дефектов и улучшения процесса – система должна предоставлять статистику по типам и частоте дефектов, сменам, операторам и участкам контроля, чтобы технологи могли выявлять тренды, причины отклонений и заниматься снижением дефектности.

5. Стандартизировать классификацию дефектов и правила оценки – система должна закрепить единый справочник типов дефектов и единые процедуры подтверждения и отклонения, чтобы обеспечить единый подход к ведению работы отделом контроля качества на разных сменах и участках. Основные типы дефектов, которые должна обнаруживать и классифицировать система: окалина (rolled-in scale, RS), пятна (patches, Pa), трещины (crazing, Cr), шероховатая поверхность (pitted surface, PS), включения (inclusion, In) и царапины (scratches, Sc)

6. Обеспечить управляемое повышение качества автоматического контроля и адаптацию к новым дефектам – система должна позволять предприятию по мере необходимости улучшать точность распознавания на данных реального производства и учитывать появление новых типов дефектов (за счет расширения классификации и последующего улучшения распознавания), без остановки основных процессов контроля качества.

3.3 Выявление требований пользователя

Для выявления требований пользователей была проведена вымышленная беседа работников предприятия с участка контроля качества, среди которых есть операторы ОТК, технологи и начальники-администраторы.

Интервьюер: Мальцев Максим Александрович

- Сотрудник 1:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь?

Ответ на вопрос: Я оператор ОТК. Проверяю листы металла после прокатки и принимаю решение – годен лист или брак.

Вопрос № 2: Что вам было бы удобно иметь в новой системе?

Ответ на вопрос: Мне было бы удобно сразу видеть, какие дефекты система нашла на листе. Чтобы они были подсвечены, и я мог быстро понять и подтвердить – согласен я или нет (брак ли это, или нет). Ещё хочу иметь возможность исправить найденные дефекты, если вдруг что-то пойдет не так, – изменить тип или удалить лишние.

Вопрос № 3: Нужна ли вам история проверок и своя статистика?

Ответ на вопрос: Да, конечно. Мне было бы удобно видеть список всех проверенных и непроверенных листов, чтобы можно было быстро вернуться к какому-то случаю. А моя статистика даст мне понять, как я работаю (сколько листов, например, обработал).

- Сотрудник 2:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь?

Ответ на вопрос: Я младший технолог. Занимаюсь анализом дефектности и причинами отклонений качества.

Вопрос № 2: Что для вас важно в новой системе?

Ответ на вопрос: Мне было бы удобно видеть общую статистику по типам дефектов за период – чтобы понимать, какие проблемы преобладают. Ещё хорошо было бы, если бы я также видел историю всех проверок, мог перейти к какой-нибудь, и видел, кто отметил дефект и в какой зоне, это мне нужно. А ещё статистика за каждого пользователя и зону ответственности.

- Сотрудник 3:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь?

Ответ на вопрос: Я начальник участка контроля качества и по совместительству будущий администратор этой системы. Отвечаю за доступы и техническую часть.

Вопрос № 2: Что для вас важно в новой системе?

Ответ на вопрос: Мне было бы удобно управлять учетными записями – создавать пользователей, назначать роли, закреплять их за участками. Также я буду следить за подключением камер, мне нужно иметь возможность их настраивать и видеть, что данные поступают корректно.

Вопрос № 3: Нужно ли управлять моделью распознавания?

Ответ на вопрос: Да, главный технолог и глава говорили, что система должна иметь возможность подстраиваться под обнаружение дефектов, которые могут появиться и которые мы захотим отдельно классифицировать в будущем. Так что хорошо было бы обновлять модель без остановки системы, в возможностью отката, если вдруг пойдет что-то не так. Под обновлением я подразумеваю дообучение модели на дефектах, которые обнаружат мои операторы (причем как на уже существующем, так и на предсказании нового класса).

Полученное интервью-опрос можно использовать для документирования требований пользователей. Для их представления было принято решение использовать пользовательские истории (User Story) для трёх ролей:

1) Оператор ОТК – главный и наиболее частый пользователь системы, чья задача принимать правильные решения по листу металла (годен или брак) на основании его внешнего вида (по наличию видимых дефектов).

- КАК оператор, Я ХОЧУ получать результаты автоматического контроля по листу металла, ЧТОБЫ быстрее принимать решение о качестве продукции.

- КАК оператор, Я ХОЧУ верифицировать результаты автоматического обнаружения дефектов (подтверждать или отклонять) и при необходимости корректировать сведения о дефектах и их типах, ЧТОБЫ обеспечить достоверность итогового решения и снизить риск пропуска брака.

- КАК оператор, Я ХОЧУ видеть историю всех поступивших изображений и результатов их проверки, ЧТОБЫ быстро находить нужные случаи, исправлять их при необходимости или верифицировать необработанные изображения.

- КАК оператор, Я ХОЧУ видеть свою статистику работы (объем проверок, доли дефектов или исправлений), ЧТОБЫ понимать качество и эффективность своей работы, а также не перерабатывать.

2) Технолог – человек, чья задача снижать дефектность, находить причины отклонений качества на основе данных контроля (тип обнаруженного дефекта, место, где дефект был обнаружен, работник, обнаруживший его) и стандартизировать подход к оценке.

- КАК технолог, Я ХОЧУ видеть сводную статистику по типам обнаруженных дефектов за период, ЧТОБЫ понимать структуру дефектности и приоритеты улучшений.

- КАК технолог, Я ХОЧУ анализировать дефектность в разрезе смен, операторов и участков контроля, ЧТОБЫ выявлять отклонения процесса и влияющие факторы.

- КАК технолог, Я ХОЧУ проваливаться из статистики к конкретным листам (изображениям) и решениям оператора, ЧТОБЫ разбирать причины и подтверждать выводы фактами.

- КАК технолог, Я ХОЧУ видеть историю всех поступивших изображений и результатов их проверки с возможностью фильтрации, ЧТОБЫ быстро находить нужные для анализа случаи.

3) Администратор – человек, цель которого обеспечить работоспособность, безопасность и управляемость системы на предприятии, а также поддержание развития качества распознавания и классификации дефектов.

- КАК администратор, Я ХОЧУ управлять учетными записями сотрудников, создавать их и назначать им роль и участок (зону ответственности), ЧТОБЫ обеспечить безопасность, корректный доступ к функциям системы и разграничение ответственности.

- КАК администратор, Я ХОЧУ настраивать параметры подключения и работы источников изображений (камер на линиях металлопроката), ЧТОБЫ система стабильно получала правильные данные для контроля.

- КАК администратор, Я ХОЧУ управлять справочником типов дефектов (включая добавление новых при их появлении), ЧТОБЫ поддерживать актуальную классификацию предприятия.

- КАК администратор, Я ХОЧУ иметь возможность по необходимости проводить улучшение качества распознавания на данных предприятия, ЧТОБЫ система адаптировалась к реальным условиям и новым дефектам управляемым образом.

- КАК администратор, Я ХОЧУ управлять эксплуатационными параметрами системы и её версиями, ЧТОБЫ поддерживать стабильную работу без сбоев на производстве и производить откат в случае ухудшения качества работы модели.

3.4 Составление списка функциональных требований

Функциональные требования были получены декомпозицией бизнес-требований и требований пользователей. Список функциональных требований:

FRG-1. Управление доступом и ролями:

- FRQ-1.1 Система должна предоставлять пользователю возможность аутентифицироваться по логину и паролю.

- FRQ-1.2 Система должна разграничивать доступ к функциям по ролям: оператор, технолог, администратор.

- FRQ-1.3 Система должна обеспечивать назначение пользователю зоны ответственности (участка) и учитывать её при работе пользователя в системе.

FRQ-2. Управление пользователями (администратор):

- FRQ-2.1 Система должна предоставлять администратору возможность создавать учетную запись пользователя.

- FRQ-2.2 При создании учетной записи система должна позволять администратору задавать пользователю роль и участок (зону ответственности).

- FRQ-2.3 Система должна предоставлять администратору возможность изменять роль пользователя.

- FRQ-2.4 Система должна предоставлять администратору возможность удалять пользователя.

- FRQ-2.5 Система должна предоставлять администратору возможность создавать участки (зоны ответственности).

FRQ-3. Получение и хранение изображений:

- FRQ-3.1 Система должна принимать изображения листов металла от источника (камеры), связанного с участком (линией).

- FRQ-3.2 Система должна сохранять все поступившие изображения в хранилище/базе данных.

- FRQ-3.3 Для каждого поступившего изображения система должна создавать запись контроля, связывающую изображение, результаты детекции и последующие действия пользователя.

FRQ-4. Автоматическая детекция дефектов:

- FRQ-4.1 Система должна выполнять предобработку изображения перед детекцией дефектов (приведение к виду обучающих данных).

- FRQ-4.2 Система должна выполнять детекцию дефектов на изображении и формировать результат в виде набора объектов «дефект» (как минимум: тип, координаты области/прямоугольника, уверенность).

- FRQ-4.3 Система должна поддерживать автоматическое обнаружение и классификацию дефектов типов: окалина (RS), пятна (Pa), трещины (Cr), шероховатая поверхность (PS), включения (In) и царапины (Sc).

- FRQ-4.4 Система должна уметь помечать дефект типом «другое» (прочее) для случаев, не соответствующих известным типам.

FRQ-5. Статусы обработки изображения:

- FRQ-5.1 Система должна присваивать изображению статус, отражающий этап обработки, включая как минимум: pending (ожидает подтверждения), accepted (годен), rejected (брак).

- FRQ-5.2 Для нового загруженного в сеть изображения листа устанавливается статус pending.

FRQ-6. Верификация результатов оператором:

- FRQ-6.1 Система должна предоставлять оператору возможность выбрать изображение для верификации.

- FRQ-6.2 Система должна отображать оператору изображение и результаты автоматической детекции - список обнаруженных дефектов.

- FRQ-6.3 Система должна предоставлять оператору возможность подтвердить обнаруженные дефекты.

- FRQ-6.4 Система должна предоставлять оператору возможность отклонить обнаруженные дефекты.

- FRQ-6.5 Система должна предоставлять оператору возможность установить итоговый статус продукта/изображения «годен» или «брак» (через accepted/rejected).

FRQ-7. Ручная корректировка разметки дефектов (оператор):

- FRQ-7.1 Система должна предоставлять оператору возможность исключать некоторые дефекты на выбор из результатов автоматической детекции при необходимости.

- FRQ-7.2 Система должна предоставлять оператору возможность добавлять новый дефект вручную, выделяя его область на изображении.

- FRQ-7.3 При добавлении дефекта вручную система должна предоставлять оператору возможность указать тип дефекта из набора меток (включая «другое»).

- FRQ-7.4 Система должна сохранять все внесенные оператором изменения разметки вместе с авторством и временем изменений.

FRQ-8. История изображений и фильтрация:

- FRQ-8.1 Система должна предоставлять пользователю возможность просматривать историю поступивших изображений и результатов их обработки.

- FRQ-8.2 Система должна предоставлять фильтрацию истории по статусу (accepted/rejected/pending).

- FRQ-8.3 Система должна предоставлять фильтрацию истории по наличию дефектов.

- FRQ-8.4 Система должна предоставлять фильтрацию истории по типам дефектов.

- FRQ-8.5 Система должна предоставлять фильтрацию истории по периоду.

- FRQ-8.6 Система должна предоставлять фильтрацию истории по участку (зоне ответственности или камере).

- FRQ-8.7 Система должна предоставлять фильтрацию истории по пользователю.

FRQ-9. Статистика и аналитика:

- FRQ-9.1 Система должна собирать статистику по работе операторов и по обработанным изображениям.

- FRQ-9.2 Система должна предоставлять оператору просмотр собственной статистики (включая как минимум: количество обработанных изображений, количество дефектов, среднее время до верификации).

- FRQ-9.3 Система должна предоставлять технологу просмотр общей статистики по всем операторам и изображениям.

- FRQ-9.4 Система должна предоставлять технологу просмотр статистики по выбранному оператору.

- FRQ-9.5 Система должна предоставлять выбор периода для просмотра статистики.

- FRQ-9.6 Система должна предоставлять переход от агрегированной статистики к конкретным изображениям и решениям оператора (для анализа причин).

FRQ-10. Настройка камеры (администратор):

- FRQ-10.1 Система должна предоставлять администратору возможность настраивать параметры камеры/источника изображений.

- FRQ-10.2 Система должна предоставлять изменение разрешения снимков.

- FRQ-10.3 Система должна предоставлять изменение интервала/частоты снимков.

FRQ-11. Настройка модели детекции и развитие классификации (администратор):

- FRQ-11.1 Система должна предоставлять администратору возможность задавать порог уверенности детекции.

- FRQ-11.2 Система должна предоставлять администратору возможность расширять справочник типов дефектов (добавлять новую метку/тип).

- FRQ-11.3 Система должна предоставлять администратору возможность группировать изображения с выделенными операторами дефектами, отмеченные как «другое», для последующего уточнения классификации.

- FRQ-11.4 Система должна предоставлять администратору возможность инициировать дообучение распознавания на выбранном наборе данных предприятия.

FRQ-12. Управление версиями модели/системы (администратор):

- FRQ-12.1 Система должна поддерживать хранение информации о версиях используемой модели детекции.

- FRQ-12.2 Система должна предоставлять администратору возможность переключения версии (ввода обновления) и отката к предыдущей версии при ухудшении качества.

3.5 Составление списка нефункциональных требований

NFRQ-1. Безопасность:

- NFRQ-1.1 Система должна обеспечивать ролевое разграничение доступа (оператор/технолог/администратор) ко всем данным и функциям.

- NFRQ-1.2 Система должна хранить учетные данные пользователей безопасным способом.

- NFRQ-1.3 Система должна обеспечивать защищенную передачу данных при доступе через сеть предприятия (например, TLS для веб-доступа).

NFRQ-2. Удобство использования:

- NFRQ-2.1 Интерфейс системы должен быть веб-ориентированным и доступным в типовых современных браузерах (на Chromium и Gecko).

- NFRQ-2.2 Интерфейс должен быть понятным для пользователей производственных ролей и обеспечивать выполнение ключевых сценариев без длительного обучения.

NFRQ-3. Сопровождаемость и управляемость изменений:

- NFRQ-3.1 Система должна поддерживать управляемое обновление модели распознавания и откат к предыдущей версии для снижения рисков деградации качества.

3.6 Выбор архитектурного паттерна

Дальше необходимо было выбрать архитектурный паттерн для данной системы. Выбранный паттерн и обоснование выбора были описаны в виде ADR (Architecture Decision Record):

Запись принятого решения №1: Выбор архитектурного паттерна для системы

Дата: 2026-03-05

Контекст:

Разрабатывается система MetalDefectTracker (MDT) – система автоматизированного контроля качества поверхности листового металла для металлопрокатного предприятия в виде веб-приложения.

Выделены главные функциональные границы: аутентификация и управление доступом, приём изображений с камер, автоматическая детекция дефектов (ML), верификация и разметка оператором, аналитика и история, администрирование (камеры, модель, пользователи).

Нефункциональные требования определяют выбор архитектурного паттерна, поскольку они задают ограничения и требования:

1) Система должна иметь невысокую операционную сложность для того, чтобы небольшая команда могла на предприятии могла сопровождать её;

2) ML-модель детекции дефектов должна развертываться отдельно, чтобы она могла обновляться независимо от остальной системы, поддерживать дообучение и откат к предыдущей версии без остановки основного процесса контроля.

Решение:

Принято решение использовать архитектуру на основе сервисов, которая будет состоять из веб-приложения и ML-сервиса. Веб-приложение в таком случае будет являться модульным монолитом, у которого будут шесть модулей, каждый из которых управляет своей зоной ответственности:

1) Accounts – аутентификация, пользователи, роли и участки (FRQ-1, FRQ-2), RBAC для NFRQ-1;

2) Receiver – приём изображений, создание записи контроля (FRQ-3);

3) Detection – HTTP-клиент к ML-сервису, сохранение результатов, статусы (FRQ-4);

4) Verification – верификация оператором, ручная разметка, аудит действий (FRQ-5, FRQ-6, FRQ-7);

5) Analytics – статистика операторов, история изображений, фильтрация (FRQ-8, FRQ-9);

6) Administration – админ панель, настройка камер, справочник дефектов, версии модели (FRQ-10, FRQ-11, FRQ-12).

Все этим модули работают с одной монолитной базой данных.

Последствия:

Преимущества:

1) Простота сопровождения. Небольшое количество сервисов легче поддерживать, а модульный монолит прост в реализации инкрементных изменений и функций пригодностей;

2) Все хранимые данные находятся в одной базе данных, что обеспечивает их целостность, и как следствие, аналитическому модулю легче агрегировать информацию;

3) Независимый цикл обновления ML. Такая архитектура позволяет обновлять модель без остановки системы;

4) Чётко определенные модули в монолите могут быть вынесены в отдельные сервисы по мере роста системы.

Недостатки и возможные риски:

1) Одна общая база данных может не справиться при высокой нагрузке, но для одного предприятия заказчика эта проблема маловероятна;

2) Единая точка отказа. При возникновении какой-либо ошибки в веб-приложении останавливается весь функционал.

В остальном, данная архитектура практична для нашего заказчика.

Альтернативы:

В качестве альтернативы рассматривалась микросервисная архитектура (превратить каждый модуль монолита в отдельный сервис со своей БД). Такой подход делает систему менее связанной и даёт возможности масштабирования, а также делает систему более отказоустойчивой. Но такой подход значительно усложняет инфраструктуру и разработку модулей, которые сильно связаны друг с другом по данным.

Также в качестве альтернативы модульному монолиту веб-приложения рассматривалась слоистая архитектура. Она проще в реализации, по сравнению

с модульной, при этом вся функциональность становится почти такой же. Однако, логика работы выделенных главных модулей становится размазанной по всем слоям одновременно, что усложняет поддержку кода и реализацию инкрементных изменений для конкретных доменов.

3.7 Реализация карты контекстов по методологии DDD

Далее данный проект был проанализирован с точки зрения методологии DDD.

Главный домен (бизнес-домен) данной системы – это автоматизированный контроль качества поверхности листового металла.

Для него были выделены поддомены:

а) Смысловое ядро:

- 1) *Выявление, верификация и разметка дефектов.* Это ключевая бизнес-ценность системы, которая будет обеспечиваться автоматизацию контроля качества.

б) Вспомогательные поддомены:

- 1) *Приём изображений и регистрация записи контроля.*
- 2) *Аналитика и история контроля.*
- 3) *Администрирование контроля.*

в) Поддомены общего назначения:

- 1) *Управление учётными записями и ролями.* Для реализации аутентификации и RBAC можно использовать уже готовые решения.

Исходя из данных поддоменов и в соответствие выбранной архитектуре были определены ограниченные контексты и связи между ними, из которых была составлена карта контекстов, показанная на рисунке 3.

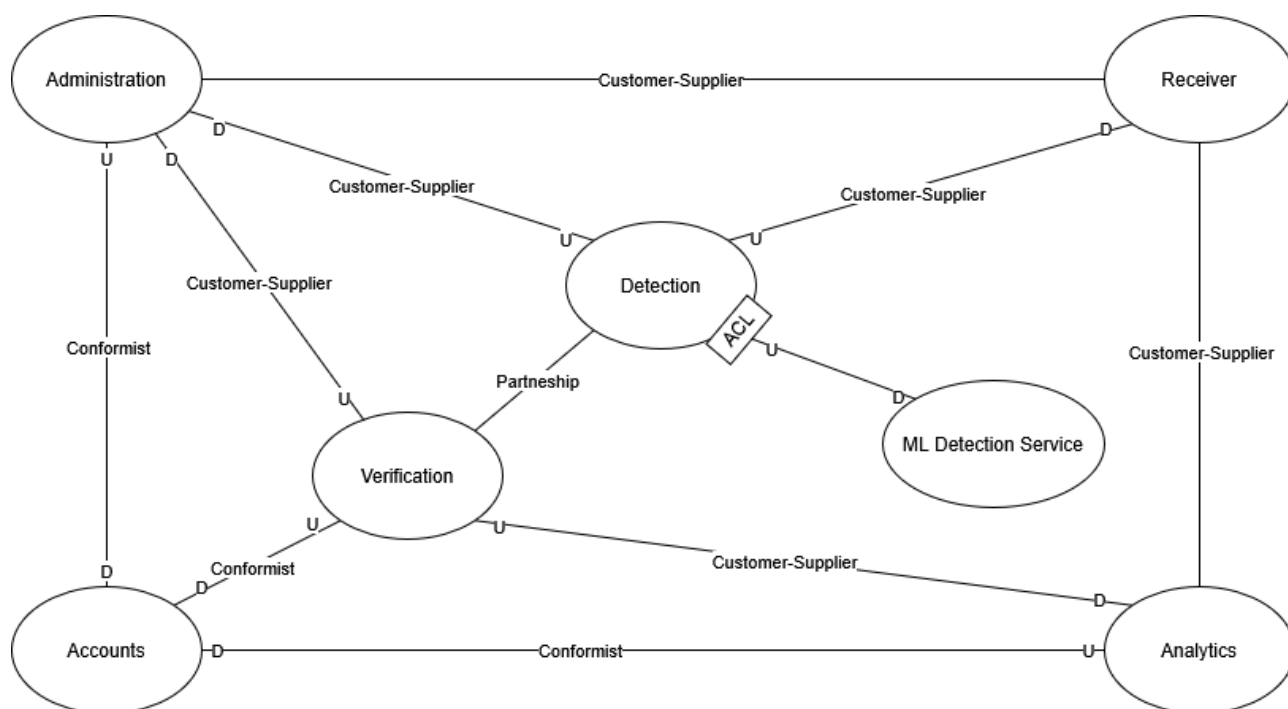


Рисунок 3 – Карта контекста системы по методологии DDD

Ограниченные контексты, приведённые на карте контекстов:

1) **Verification** и **Detection** – это два контекста, полученных декомпозицией смыслового ядра, они имеют высокую степень связанности друг с другом, поэтому они связаны типом Partnership. Это два самых главных контекста, поэтому для других он является Upstream.

2) **Accounts** – контекст аутентификации, пользователей, ролей и участков. Он задаёт единую и общую модель доступа, которую используют другие контексты, поэтому задаётся связь Conformist;

3) **Receiver** – контекст приёма изображений с камер и создания записи контроля. Он поставяет исходные данные для последующей обработки, поэтому выступает в роли поставщика для Detection, что соответствует связи Customer-Supplier;

4) **ML Detection Service** – внешним ML-сервис, в котором будут даваться предсказания полученной фотографии, для связи с Detection логично использовать Anti-Corruption Layer, так как Detection не должен зависеть от реализации сервиса;

5) **Verification** – контекст операторской верификации и ручной разметки. Здесь оператор подтверждает, дополняет или отклоняет найденные дефекты на полученной фотографии;

6) **Analytics** – контекст истории изображений, фильтрации и статистики по операторам, участкам и результатам обработки. Он агрегирует данные из других контекстов, поэтому для Receiver и Verification он является клиентом и связан с ними с помощью Customer-Supplier;

7) **Administration** – контекст администрирования системы: настройка камер, справочник типов дефектов, параметров и версий модели. Он задаёт правила и настройки для Verification, Detection (справочник типов дефектов) и Receiver (FPS камеры и др.), поэтому связан с ними связью Customer-Supplier.

3.8 Диаграмма контекста нотации C4

Для смыслового ядра и одного из вспомогательных поддоменов делаем диаграмму контекста нотации C4. Первая диаграмма показана на рисунке 4.

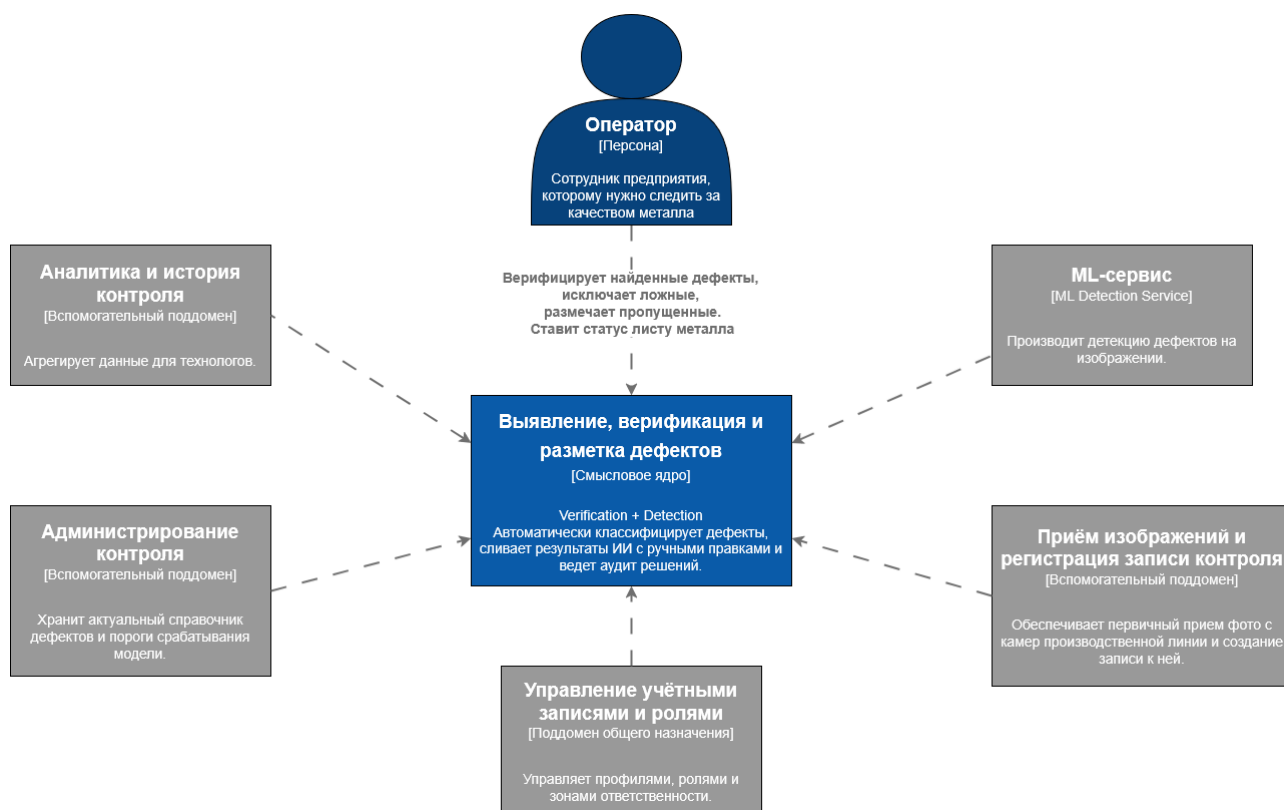


Рисунок 4 – Диаграмма контекста нотации C4 для смыслового ядра системы

Центральной системой в рамках данной диаграммы контекста – это смысловое ядро. Его основным пользователем является оператор, который получает результаты автоматической детекции, подтверждает или отклоняет найденные дефекты и вручную размечает пропущенные. Данный поддомен имеет высокую степень взаимодействия с другими поддоменами, поэтому на диаграмме описано взаимодействие с каждым из них. Помимо поддоменов, для выполнения автоматического распознавания ядро также взаимодействует с внешним ML-сервисом детекции, которому он отправляет изображения и получает результаты детекции.

На рисунке 5 показана диаграмма контекста нотации C4 для вспомогательного поддомена «Приём изображений и регистрация записи контроля».

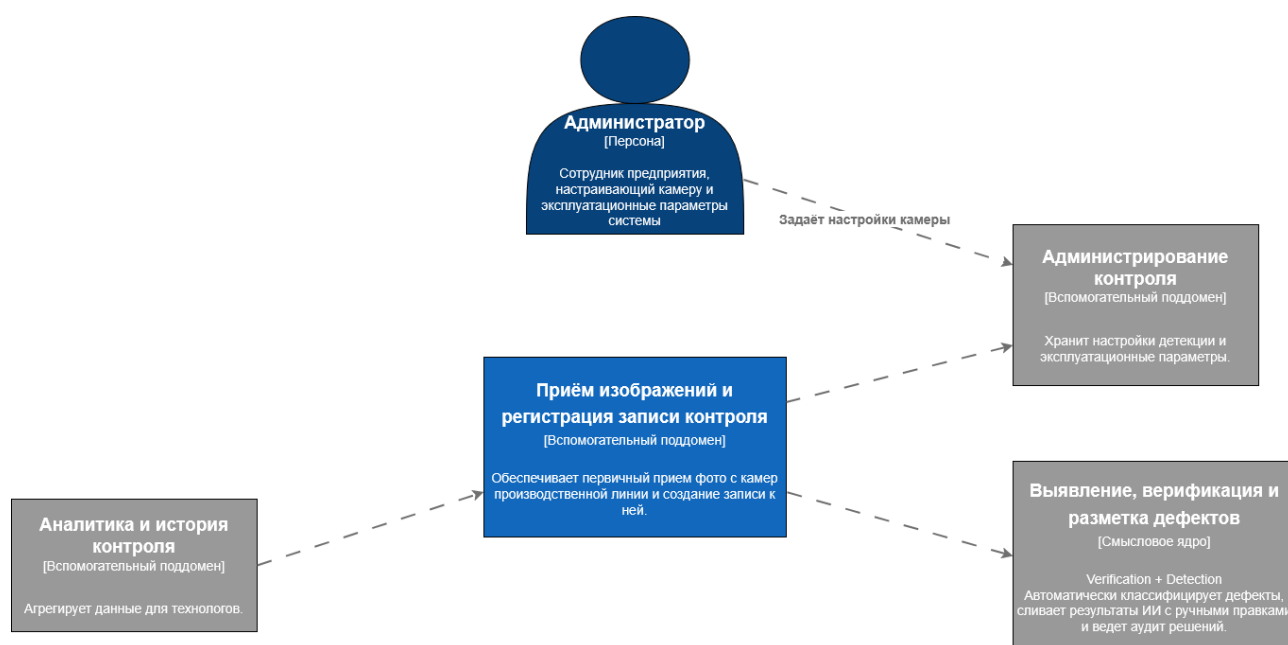


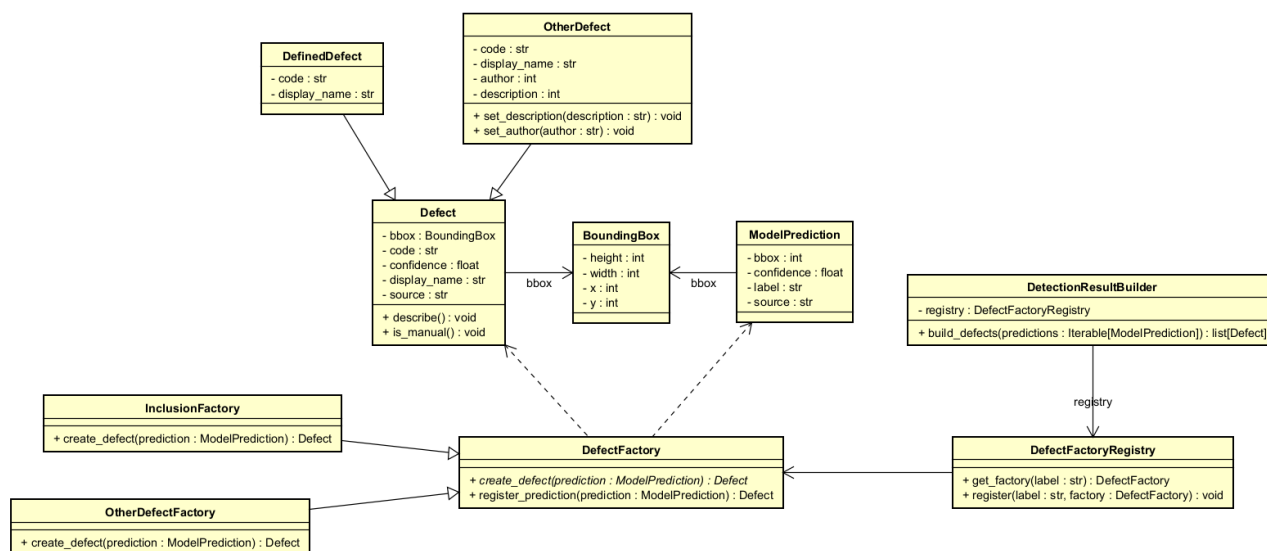
Рисунок 5 – Диаграмма контекста нотации C4 для вспомогательного поддомена системы

Центральной системы в рамках данной диаграммы – это наша вспомогательная подобласть, которая получает изображения от внешнего источника – камеры, связанной с производственным участком (линией), и создаёт запись для полученного изображения. Данным поддоменом косвенно управляет администратор. Полученное с камеры с заданными настройками

изображение сохраняется, для него создаётся запись, и вместе они передаются в смысловое ядро. Также поддомен аналитики использует этот поддомен для сбора истории изображений.

3.9 Реализация паттернов проектирования

3.9.1 Порождающий паттерн



Он применён в контексте Detection для создания доменных объектов дефектов, которые затем можно использовать дальше в коде. Абстрактный класс DefectFactory задаёт интерфейс создания объекта Defect, который также выступает в роли абстрактного продукта для конкретных типов дефектов (DefinedDefect – дефект с документированным кодом, и OtherDefect – дефект, созданный пользователем, для которого нет кода в списке дефектов), а конкретные фабрики (DefinedDefectFactory, OtherDefectFactory и др.) создают эти соответствующие конкретные типы дефектов. Клиентом паттерна выступает

DetectionResultBuilder, который преобразует список предсказаний модели в список доменных объектов.

Выбор данного паттерна обоснован тем, что система должна поддерживать несколько типов дефектов, единообразное создание объектов дефектов и возможность расширения классификации без изменения клиентского кода.

Порождающий паттерн «Абстрактная фабрика» в данном месте использовать нецелесообразно, поскольку в рассматриваемом фрагменте системы не требуется создавать семейства взаимосвязанных объектов (разные дефекты никак не связаны друг с другом).

3.9.2 Структурный паттерн

Для реализации структурного паттерна в смысловом ядре системы выбран паттерн «Адаптер». Его UML диаграмма классов показана на рисунке 7.

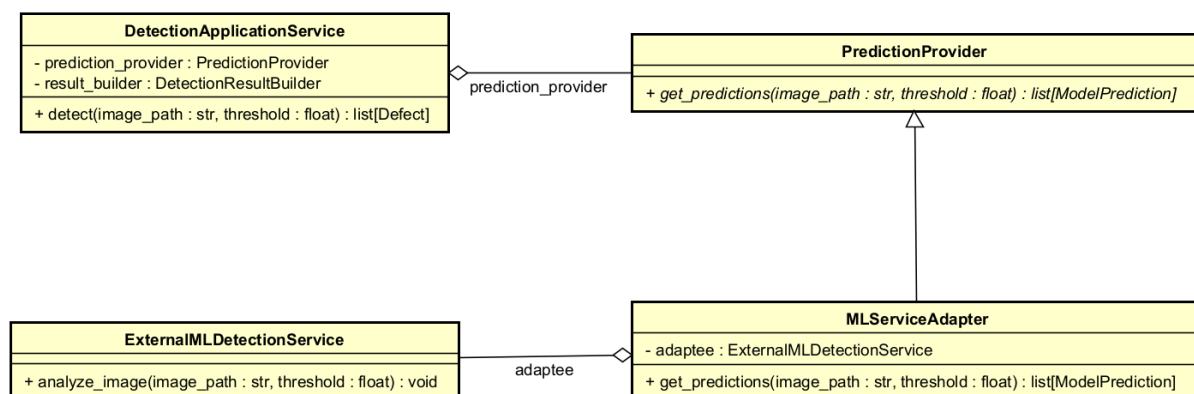


Рисунок 7 – Диаграмма классов реализации паттерна «Адаптер»

Он применён в контексте Detection, который по архитектуре взаимодействует с внешним ML Detection Service через антикоррупционный слой. Внешний сервис возвращает результаты распознавания в собственном формате (например, JSON), не совпадающем с внутренней моделью смыслового ядра. Поэтому введён интерфейс PredictionProvider, который используется внутренним сервисом DetectionApplicationService, и класс MLServiceAdapter, адаптирующий интерфейс внешнего клиента ExternalMLDetectionClient к внутреннему формату ModelPrediction.

Выбор данного паттерна обоснован необходимостью изолировать смысловое ядро от формата и технических деталей внешнего ML-сервиса, а также обеспечить слабую связанность и удобство замены внешнего поставщика распознавания.

Структурный паттерн «Компоновщик» в данном месте использовать нецелесообразно, поскольку в рассматриваемом фрагменте системы отсутствует древовидная структура объектов.

3.9.3 Поведенческий паттерн

Для реализации поведенческого паттерна в смысловом ядре системы выбран паттерн «Команда». Его UML диаграмма классов показана на рисунке 8.

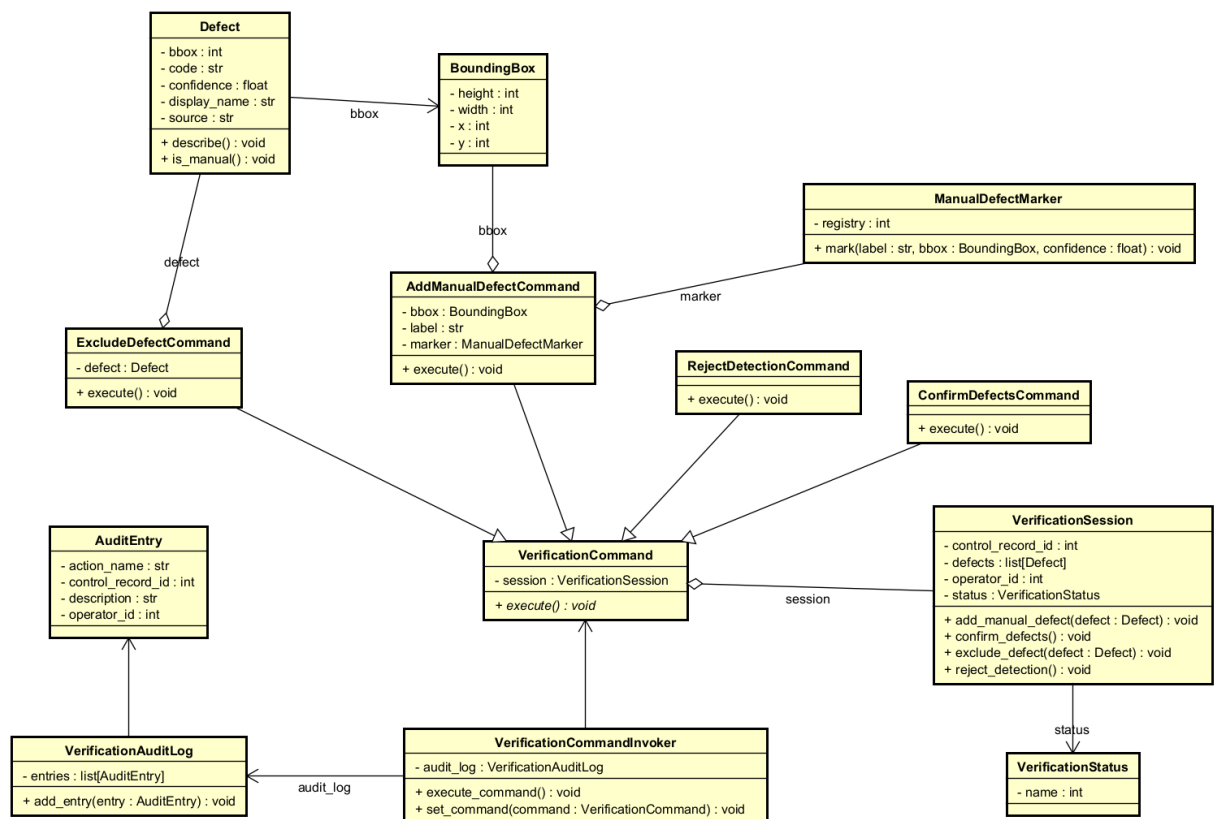


Рисунок 8 – Диаграмма классов реализации паттерна «Команда»

Он применён в контексте *Verification*, поскольку оператор системы выполняет набор действий над результатами автоматической детекции (добавление дефекта, удаление, подтверждение и отклонение). Каждое такое

действие представлено отдельным объектом-командой (AddManualDefectCommand, ExcludeDefectCommand, ConfirmDefectsCommand и RejectDefectsCommand) реализующим общий интерфейс VerificationCommand. Получателем команд выступает VerificationSession, а единым вызывающим объектом (отправителем) – VerificationCommandInvoker, который также записывает сведения об исполнении команд в журнал аудита.

Выбор паттерна обоснован необходимостью единообразно представлять действия оператора, централизованно выполнять их и фиксировать историю изменений при верификации результатов контроля.

Поведенческий паттерн «Посредник» в данном месте использовать нецелесообразно, поскольку в рассматриваемом фрагменте системы отсутствует сложная сеть взаимодействий между множеством равноправных объектов, требующая централизованной координации.

4 Выводы

По результатам выполнения практической работы была выбрана для проектирования система MetalDefectTracker (MDT). Для визуализации взаимодействия пользователей с системой была построена UML-диаграмма вариантов использования.

По результатам выполнения практической работы были составлены и проанализированы требования для проектируемой системы на всех уровнях: бизнес-требования, требования пользователей, функциональные и нефункциональные требования.

В реальной жизни источниками бизнес-требований для системы MDT могли бы выступать стратегические цели предприятия по снижению брака и повышению эффективности контроля качества, технологические регламенты и стандарты (ГОСТы), договорные обязательства перед клиентами по качеству продукции, экономические показатели и т.д.. Узнать о целях и обязательствах компании можно с помощью интервью и опросов. Требования пользователей собирались бы непосредственно от операторов, технологов и администраторов через интервью и наблюдение за их работой на производстве. Функциональные

требования выводились бы из бизнес- и пользовательских требований путём декомпозиции и анализа.

Был выбран архитектурный паттерн, был оформлен ADR, в котором был описан выбранный паттерн, был обоснован выбор и были описаны альтернативы.

Система была разобрана по методологии DDD, были выделены её поддомены (смысловое ядро, вспомогательные поддомены и поддомены общего назначения), была составлена карта контекстов по методологии DDD и диаграмма контекста нотации C4 для смыслового ядра и одного из вспомогательных поддомена.

Были реализованы паттерны проектирования каждого вида, наиболее подходящие для данной системы.